



บทที่ 7 GRAPH ALGORITHMS

Agenda Style



๐1

นิยาม

๐2

Topological Sort

๐3

Shortest-Path Algorithms

๐4

Minimum Spanning Tree

๐5

1. Prim's Algorithm

๐6

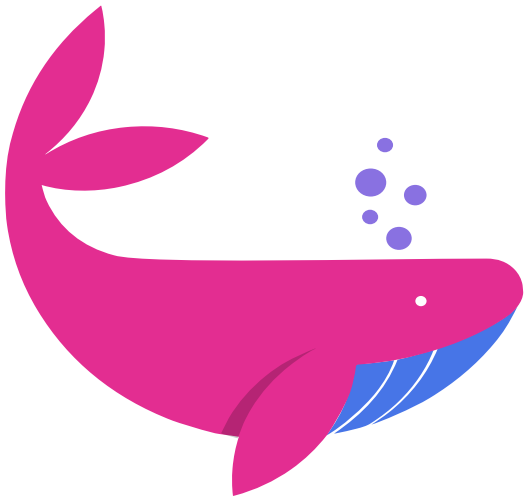
2. Kruskal's Algorithm

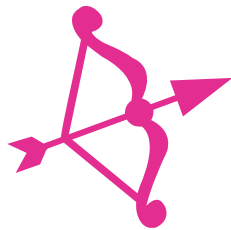
๐7

Applications of Depth-First Search

ในบทนี้จะกล่าวถึงปัญหาทั่วไปของทฤษฎีกราฟ

- กล่าวถึงปัญหาที่สามารถใช้ทฤษฎีกราฟเพื่อแก้ไขได้
- กล่าวถึง **algorithm** ในการแก้ปัญหากล่าว
- แสดงให้เห็นว่าการเลือกโครงสร้างข้อมูลที่เหมาะสมจะลด **running time** ของ **algorithm** ดังกล่าวนั้นได้มาก





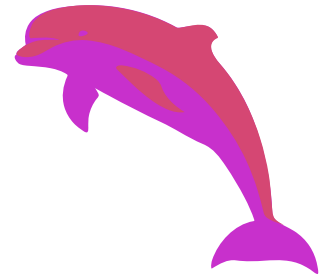
7.1 นิยาม

- **Graph $G = (V, E)$** ประกอบด้วยเซตของ *vertices* (V) และเซตของ *edges* (E)
 - แต่ละ **edge** คือคู่ของ **(v, w)** , เมื่อ **$v, w \in V$**
- ถ้า **edge** เป็นคู่อันดับก็จะได้กราฟเป็นแบบ **directed graph**
 - **Directed graphs** เรียกสั้น ๆ ว่า **digraphs**
- **Vertex w** เป็น **adjacent** กับ **v** ถ้า (**iff**) **$(v, w) \in E$**
 - สำหรับ **undirected graph** ที่มี **edge (v, w)** นั่นคือมี **(w, v)** , ดังนั้น **w** เป็น **adjacent** กับ **v** และ **v** เป็น **adjacent** กับ **w**



- บางครั้ง **edge** อาจจะมีองค์ประกอบที่เรียกว่า **weight** หรือ **cost**
- **path** คือลำดับของ **vertices** $w_1, w_2, w_3, \dots, w_n$ ที่ $(w_i, w_{i+1}) \in E$ สำหรับ $1 \leq i < n$
- **length** ของ **path** คือ จำนวน **edges** บน **path** ซึ่งเท่ากับ $n - 1$
- กรณีที่กราฟมี **edge** (v, v) จาก **vertex** ไปยังตัวมันเองแล้ว เรียก **path** v, v ว่า **loop**
- **simple path** คือ **path** ที่มี **vertices** ทั้งหมดไม่ซ้ำกัน ยกเว้นตัวแรกและตัวสุดท้ายอาจเป็นตัวเดียวกันได้

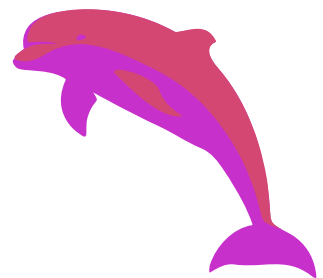




- **cycle** ใน **directed graph** คือ **path** ที่มี **length** อย่างน้อยเท่ากับ **1** ซึ่งมี $w_1 = w_n$; และ **cycle** นี้เรียกว่า **simple cycle** ถ้า **path** เป็น **simple path**
 - สำหรับ **undirected graphs** แล้วกรณีนี้ต้องเป็นคนละ **edge** กัน
- **directed graph** เป็น **acyclic** ถ้าไม่มี **cycles** และเขียนย่อ ด้วยคำว่า **DAG**



- **undirected graph** เป็นแบบ **connected** ถ้ามี **path** หนึ่งจากทุก ๆ **vertex** ไปยัง **vertex** อื่น ๆ
 - สำหรับ **directed graph** ที่มีคุณสมบัติดังกล่าวนี้ จะเรียกว่า **strongly connected**.
 - ถ้า **directed graph** ไม่เป็น **strongly connected** แต่ถ้ายกเลิกทิศทางเสียแล้วทำให้มันเป็น **connected** ได้ ก็จะใช้เรียกกราฟนี้ว่า **weakly connected**
- **complete graph** คือ **graph** ที่มี **edge** (หนึ่ง **edge**) ระหว่าง **vertices** ทุกคู่



- ระบบเส้นทางการบินเป็นตัวอย่างหนึ่งที่จำลองแบบได้ด้วย **graph**
 - สนามบินแต่ละแห่งแทนได้ด้วย **vertex** หนึ่ง และ **vertices 2 vertices** เชื่อมต่อกันด้วย **edge** หนึ่งถ้าเป็นการบินตรงระหว่างสนามบินทั้งสอง
 - แต่ละ **edge** อาจประกอบด้วย **weight** ซึ่งอาจจะหมายถึงเวลาที่ใช้ในการบิน หรือระยะทาง หรือค่าตัวก็ได้
- การจราจรก็สามารถจำลองได้ด้วย **graph**
 - จุดตัดของถนนแทนได้ด้วย **vertex** หนึ่ง และถนนแต่ละเส้นก็คือ **edge** โดยที่ **edge costs** อาจหมายถึงขีดจำกัดความเร็ว หรือจำนวนเลนวิ่งก็ได้



7.1.1. Representation of Graphs

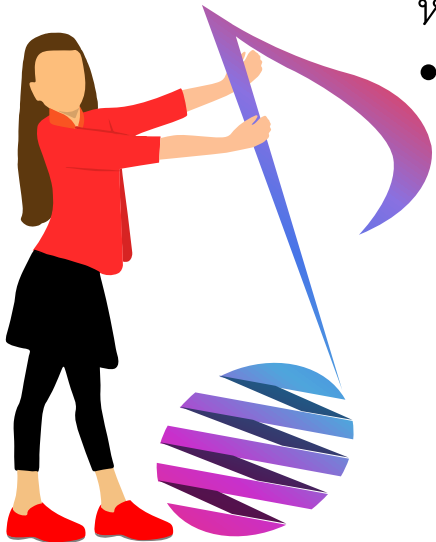
ในที่นี้จะกล่าวถึง **directed graphs** (ส่วน **undirected graphs** ก็มีลักษณะคล้ายกัน)

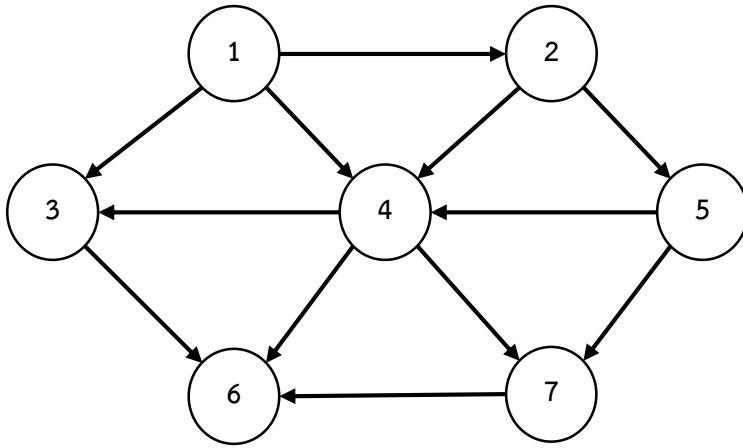
- **adjacency matrix representation** เป็น **two-dimensional array**

- สำหรับแต่ละ **edge** (u, v) , กำหนดให้ $a[u][v]$ เป็น **true**;

ถ้าไม่มี **edge** ก็ให้เป็น **false** กรณีที่มี **weight** ด้วยก็กำหนดให้ $a[u][v]$ มีค่าเท่ากับ **weight** และให้ใช้ค่า **weight** สูงมาก ๆ หรือ ต่ำมาก ๆ เป็น **sentinel** ที่หมายถึงไม่มี **edges** อยู่

- การใช้ **adjacency matrix** ต้องใช้เนื้อที่ $\Theta(|V|^2)$, ใช้กับ กราฟนั้นมี **edges** จำนวนมาก **dense graph** กล่าวคือ $|E| = \Theta(|V|^2)$





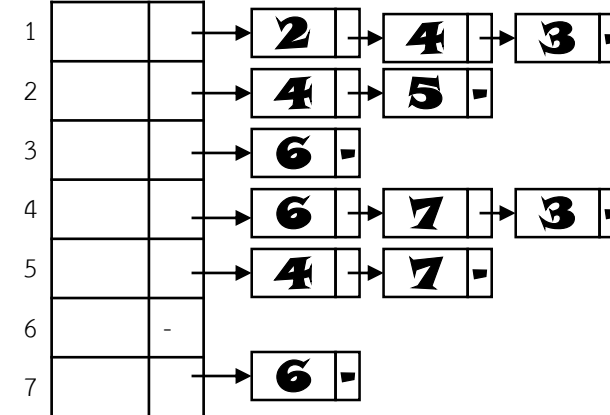
node $V = \{1, 2, 3, 4, 5, 6, 7\}$

edge $E = \{ (1, 2), (1, 4), (1, 3), (2, 4), (2, 5), (3, 6), (4, 6), (4, 7), (4, 3), (5, 4), (5, 7), (7, 6) \}$

Figure 7.1 Directed graph

0	1	1	1	0	0	0
0	0	0	1	1	0	0
0	0	0	0	0	1	0
0	0	1	0	0	1	1
0	0	0	1	0	0	1
0	0	0	0	0	0	0
0	0	0	0	0	1	0

adjacency matrix



adjacency list



adjacency list คือ

- แต่ละ **vertex** มี **list** ของ **adjacent vertices** ทั้งหมดของมัน และใช้เนื้อที่ขนาด $O(|E| + |V|)$
- โครงสร้างทางด้านซ้ายเป็นเพียง **array** ที่มี **header** เท่านั้น
- กรณีที่ **edges** มี **weights** ด้วย ก็เพียงเพิ่มเติมข้อมูลนี้ลงใน **cells** ด้วยเท่านั้น
- สำหรับ **Undirected graphs** แล้ว แต่ละ **edge (u, v)** จะปรากฏอยู่ใน **lists** สองแห่ง

การทำงานพื้นฐานที่ต้องใช้ใน **algorithm** ของกราฟ คือ การหา **vertices** ที่ **adjacent** กับ **vertex v** ที่กำหนด และสามารถทำได้โดยใช้เวลาตามจำนวน **vertices** ที่พบด้วยการใช้การ **scan** ไปตาม **adjacency list**

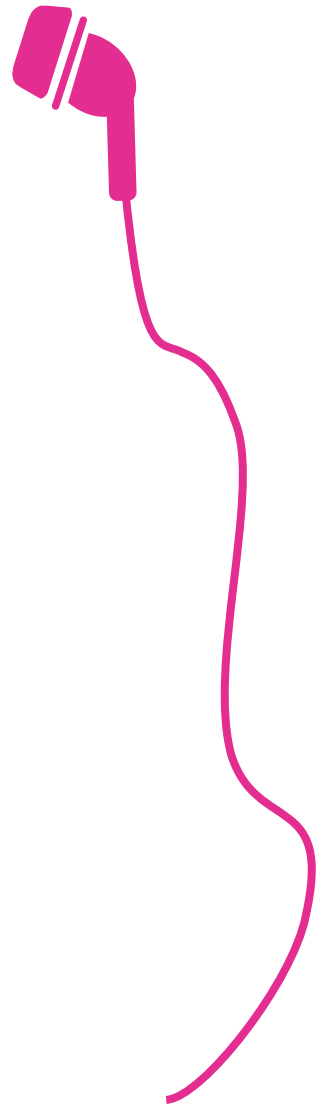
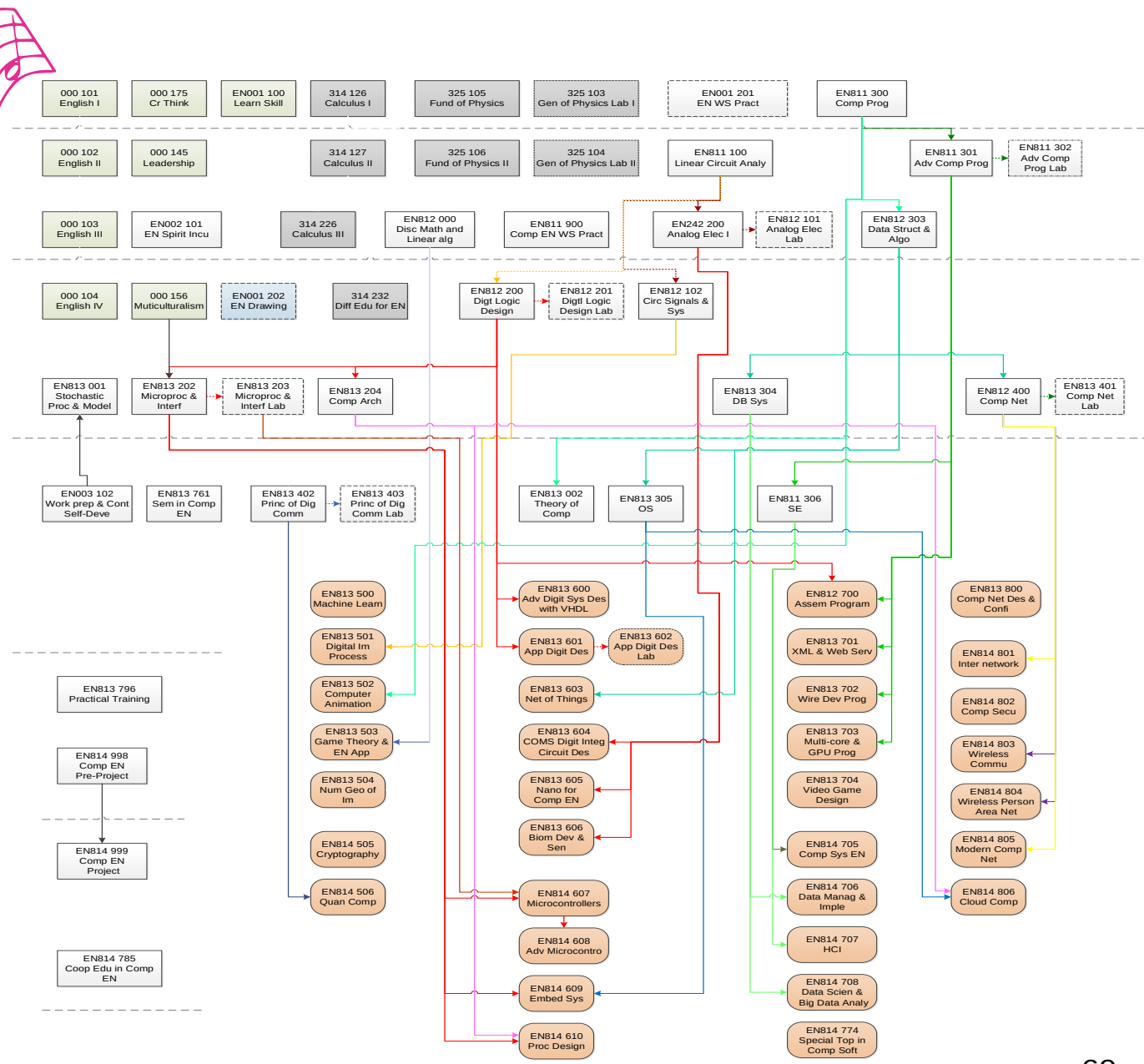




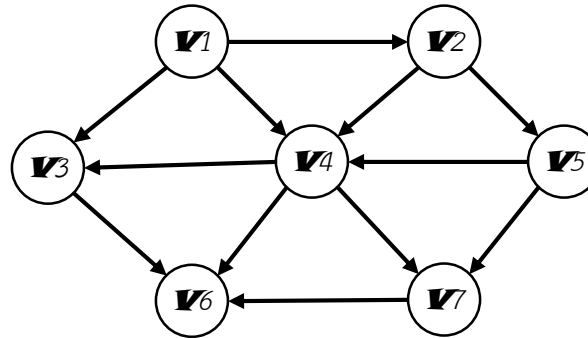
7.2. Topological Sort

- **topological sort** คือลำดับของ **vertices** ใน **directed acyclic graph** ที่ถ้ามี **path** หนึ่งจาก V_i ไปยัง V_j , แล้ว V_j มีลำดับที่ตามหลัง V_i
 - กราฟในรูป **Figure 7.3** ใช้แทนโครงสร้างความต่อเนื่องของรายวิชา **topological ordering** ของรายวิชาคือลำดับรายวิชาที่เป็นไปตามข้อกำหนดเงื่อนไขรายวิชา (**prerequisite**)

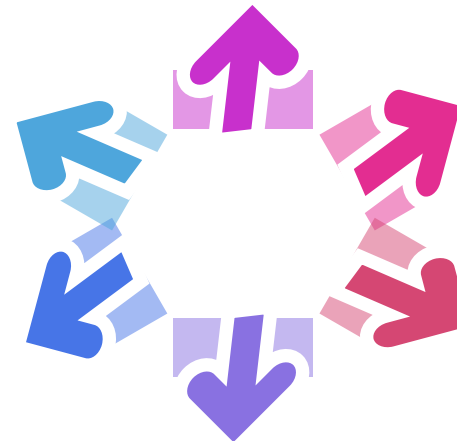




- **Topological ordering** เป็นไปไม่ได้สำหรับกราฟที่มี **cycle** เนื่องจาก **vertices** v และ w ในเส้นทาง **cycle** นั้นมี v ก่อน w และมี w ก่อน v
- **Topological ordering** ไม่จำเป็นต้องเป็น **unique**



กราฟ มี $v_1 v_2 v_5 v_4 v_3 v_7 v_6$ และมี $v_1 v_2 v_5 v_4 v_7 v_3 v_6$ ทั้งสองเป็น **topological orderings**



- **Algorithm** อย่างง่ายในการหา **topological ordering** คือ ตอนแรกหา **vertex** ที่ไม่มี **incoming edges** เป็นจุดเริ่มต้น แล้วพิมพ์ชื่อ **vertex** นี้ พร้อมกับย้ายมันพร้อมกับ **edges** ออกจากกราฟ จากนั้นก็ทำเช่นเดียวกันนี้กับส่วนที่เหลือในกราฟ
- กำหนดให้ **indegree** ของ **vertex v** คือจำนวนของ **edges (u, v)**
 - เราคำนวณค่า **indegrees** ของ **vertices** ทั้งหมดในกราฟได้ โดยถ้ามี **indegree array** และกราฟนั้นถูกจัดอยู่ใน **adjacency list** แล้ว เราก็สามารถใช้ **algorithm** ใน **Figure 7.5** เพื่อสร้าง **topological ordering** ได้



```

void topsort() throws CycleFound
{
    vertex v, w;
    for ( int counter=0; counter<NUM_VERTEX; counter++ )
    {
        v = findNewVertexOfIndegreeZero( );
        if ( v == null ) throw new CycleFound();
        v.topNum = counter;
        for each w adjacent to v
            w.indegree--;
    }
}

```

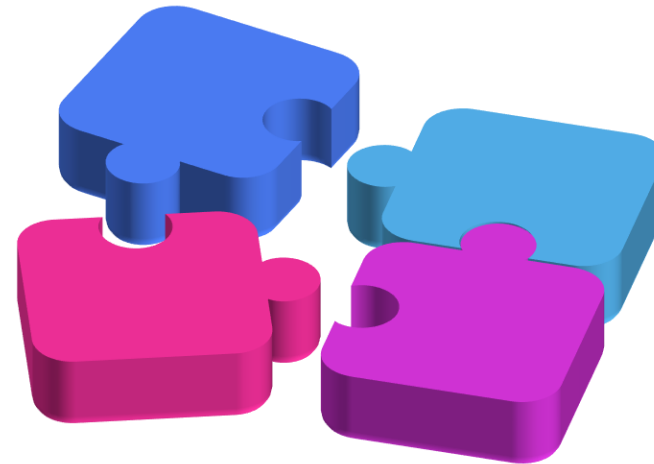
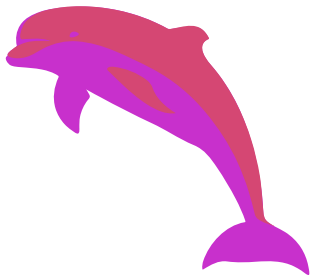


Figure 7.5 Simple topological sort pseudocode

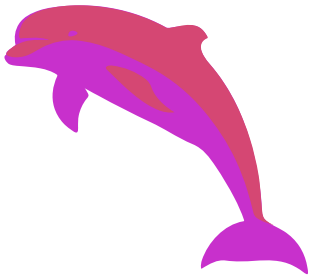
- **findNewVertexOfIndegreeZero** เป็นเพียงการ **scan** ใน **indegree array** การทำงานแต่ละครั้งใช้เวลา $O(|V|)$ และเนื่องจากการเรียกใช้เป็นจำนวน $|V|$ ครั้ง ดังนั้นมันจึงมี **running time** เป็น $O(|V|^2)$
 - สาเหตุที่มี **running time** ไม่ดีก็เนื่องจากการ **sequential scan** ใน **indegree array** นั้นเอง
- เราสามารถปรับปรุงประสิทธิภาพได้ด้วยการนำ **(unassigned) vertices** ทั้งหมดที่มี **indegree** เป็น $\bullet$ ไปไว้ในกล่องต่างหาก
 - **findNewVertexOfIndegreeZero** จะให้ค่าเป็น (และย้ายออก) **vertex** จากกล่องดังกล่าว ขณะที่เราลดค่า **indegrees** ของ **adjacent vertices** นั้นเราจะตรวจค่าของมันและย้ายมันไปไว้ในกล่องดังกล่าวเมื่อค่า **indegree** ของมันลดลงเป็น $\bullet$

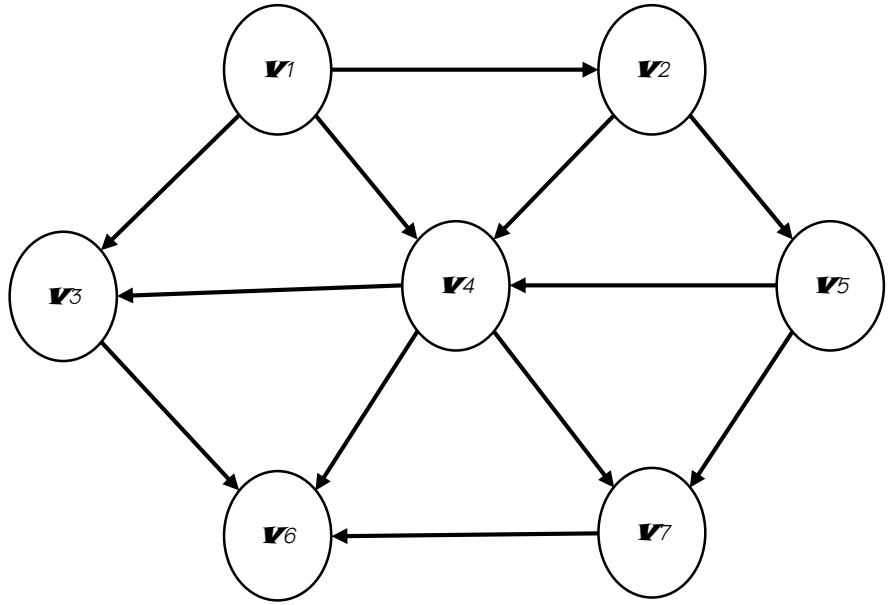


- กลองดั่งกล่าวข้างบนนั้น อาจจะใช้ **stack** หรือ **queue** ก็ได้
 - เริ่มต้นด้วยการคำนวณค่า **indegree** ของทุก ๆ **vertex**
 - จากนั้น นำ **vertices** ที่มี **indegree** เป็น 0 ทั้งหมดไปไว้ใน **queue** ซึ่งเริ่มต้นเป็น **queue** ว่าง
 - ให้ทำการย้าย **vertex v** ออกตรงเท่าที่ **queue** ยังไม่เป็น **queue** ว่าง และให้ลดค่า **indegrees** ของ **vertex** ทุกตัวที่เป็น **adjacent** กับ **v**
 - นำ **vertex** ไปเข้า **queue** เมื่อ **indegree** ของมันเป็น 0
- **topological ordering** ที่ได้ก็คือลำดับที่ **vertices** ถูก **dequeue** รูปที่ **Figure 7.6** แสดงสถานะการทำงาน



- **pseudocode** ของ **algorithm** แสดงในรูปที่ **Figure 7.7.**
 - สมมุติให้มี **adjacency list** และค่า **indegrees** ของ **vertices** เรียบร้อยแล้ว
 - ให้ **vertex** มีข้อมูลชื่อ **topNum** ซึ่งเป็น **topological number** ของมัน
- เวลาที่ใช้ในการทำงานของอัลกอริทึมคือ $O(|E| + |V|)$ ถ้าใช้ **adjacency lists** ซึ่งจะเห็นได้จาก **for loop** ที่บรรทัดที่ 8 ที่มีการทำงานเพียงหนึ่งครั้งต่อหนึ่ง **edge**
- การทำงานของ **queue** เกิดขึ้นหนึ่งครั้งต่อหนึ่ง **vertex**, และการ **initialization** ก็ใช้เวลาตามขนาดของกราฟเช่นเดียวกัน





Indegree before dequeue							
vertex	1	2	3	4	5	6	7
V1	0	0	0	0	0	0	0
V2	1	0	0	0	0	0	0
V3	2	1	1	1	0	0	0
V4	3	2	1	0	0	0	0
V5	1	1	0	0	0	0	0
V6	3	3	3	3	2	1	0
V7	2	2	2	1	0	0	0
enqueue	v1	v2	v5	v4	v3, v7	v7	v6
dequeue	v1	v2	v5	v4	v3	v7	v6

Figure 7.6 topological sort ของกราฟรูป **Figure 7.4**



```

void topSort( ) throws CycleFound
{
    Queue q; int counter;    vertex v, w;
/*1*/   q = new Queue( );
/*2*/   for each vertex v
/*3*/       if ( v.indegree == 0 )
/*4*/           q.enqueue( v );
/*5*/   while ( !q.isEmpty() )
    {
/*6*/       v = q.dequeue( );
/*7*/       v.topNum = ++counter; /* assign next number */
/*8*/       for each w adjacent to v
/*9*/           if ( --w.indegree == 0 )
/*10*/               q.enqueue( w );
    }
/*11*/   if ( counter != NUM_VERTICES )
/*12*/       throw new CycleFound();
}

```

Size of graph

At most one per vertex

Execute at most once per edge

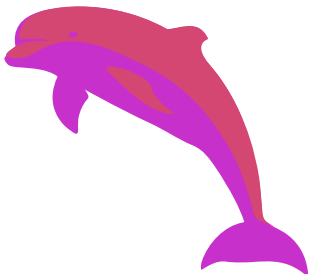
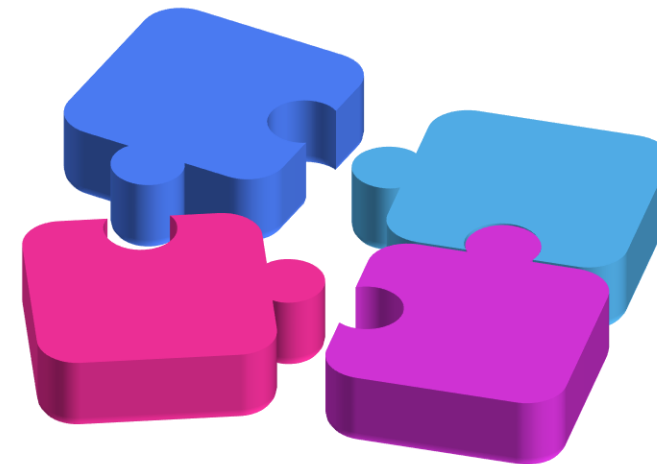


Figure 7.7 Pseudocode of topological sort

7.3. Shortest-Path Algorithms

- Input เป็น weighted graph: คือ edge (v_i, v_j) มี cost $c_{i,j}$ ในการท่องเที่ยวใน edge
- cost ของ path $v_1 v_2 \dots v_n$ คือ $\sum_{i=1}^{n-1} c_{i,i+1}$ เรียกว่า weighted path length
- unweighted path length ก็คือ จำนวน edges บนเส้นทางของ path $v_1 v_2 \dots v_n$ คือ $n - 1$



Single-source Shortest-path Problem:

input เป็น **weighted graph**, $G = (V, E)$, และมี **vertex s** ให้ทำการหา **shortest weighted path** จาก s ไปยัง **vertex** อื่น ๆ ทุก **vertex** ใน G

ปัญหาแบบอื่น

- **Single-destination shortest-paths problem:** กลับกราฟ
- **Single-pair shortest-path problem:** ปัญหาเดียวกัน
- **All-pairs shortest-paths problem:** มีวิธีเฉพาะ



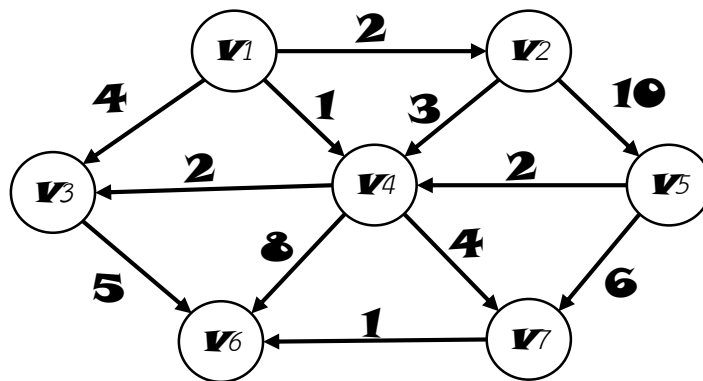


Figure 7.8 directed graph G

เช่น กราฟในรูป มี **shortest weighted path** จาก v_1 ถึง v_6

ตาม **path** = $\{ v_1 , v_4 , v_7 , v_6 \}$

มี **cost** = $1+4+1=6$

และมี **shortest unweighted path** เป็น 2

ตาม **path** = $\{ v_1 , v_4 , v_6 \}$

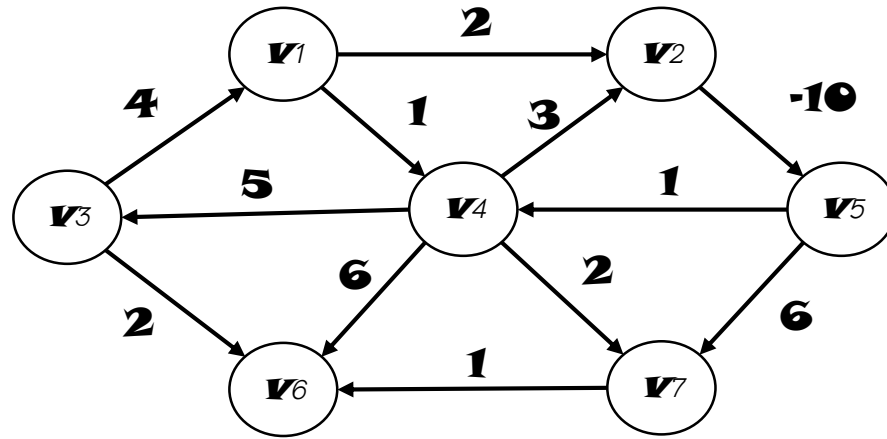


Figure 7.9 graph ที่มี **negative-cost cycle**

- ปัญหาที่เกิดจาก **negative edges**

- **path** จาก v_5 ไป v_4 มี **cost 1** แต่มี **path** ที่สั้นกว่าคือตาม **loop v_5, v_4, v_2, v_5, v_4** ซึ่งมี **cost เป็น -5** ความจริงนี้ยังไม่ใช้เส้นทางที่สั้นที่สุดเนื่องจากเราสามารถวนรอบไปได้เรื่อยๆ นั่นเอง และทำนองเดียวกับ **shortest path** จาก v_1 ไป v_6 เนื่องจากเราจะเข้าสู่การวนรอบเดียวกัน



- การวนรอบดังกล่าวนี้เรียกว่า **negative-cost cycle** ซึ่งเมื่อเกิดขึ้นในกราฟแล้วก็ไม่สามารถหา **shortest paths** ได้
 - การมี **Negative-cost edges** ไม่ใช่สิ่งที่เสียหายเหมือนกับการมี **cycles** ในกราฟ เพียงแต่มันจะทำให้การแก้ปัญหายุ่งยากขึ้น
 - เพื่อความสะดวกเราจะกำหนดให้ **shortest path** จาก **s** ไป **s** มีค่าเป็น ∞



- เราจะกล่าวถึง **algorithm** สำหรับปัญหานี้ **4** วิธี
 - ในตอนแรกจะกล่าวถึงปัญหา **unweighted shortest-path** และแก้โดยใช้เวลาเป็น $\Theta(|E| + |V|)$
 - จากนั้นแสดงการแก้ปัญหา **weighted shortest-path** โดยสมมติให้กราฟไม่มี **negative edges** และมี **running time** เป็น $\Theta(|E| \log |V|)$ เมื่อใช้โครงสร้างข้อมูลเหมาะสม
 - กรณีกราฟมี **negative edges** มีวิธีแก้ปัญหาย่างง่ายแต่ใช้เวลาเป็น $\Theta(|E| \times |V|)$
 - สุดท้ายแก้ปัญหา **weighted graph** ที่เป็น **acyclic graphs** แบบพิเศษด้วยเวลาที่เป็น **linear**



7.3.1. Dijkstra's Algorithm (Edsger Wybe Dijkstra 1930 - 2002, Netherlands)

- กรณีเป็น **weighted graph** เรายังคงใช้แนวคิดเดียวกับ **unweighted graph**
- ข้อมูลต่าง ๆ ที่ใช้ยังคงเหมือนเดิม
 - แต่ละ **vertex** ถูกกำหนดเป็น **known** หรือ **unknown**
 - เก็บค่าระยะ d_v ของแต่ละ **vertex** เช่นเดิม ระยะทางนี้จะป็นระยะทางที่สั้นที่สุดจาก s ไป v โดยใช้เฉพาะ **vertices** ต่าง ๆ ที่เป็น **known** เป็น **vertex** ระหว่างทางเท่านั้น
 - บันทึก p_v ซึ่งเป็น **vertex** สุดท้ายที่ทำให้เกิดการเปลี่ยนแปลงขึ้นกับ d_v





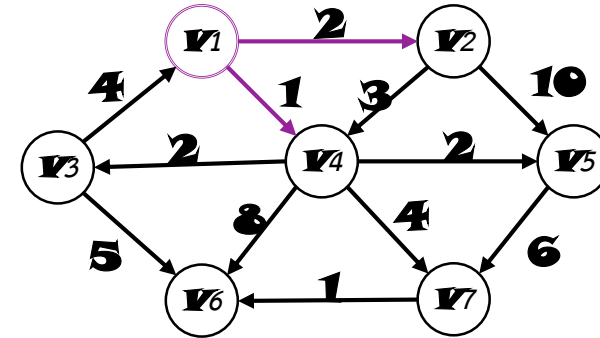
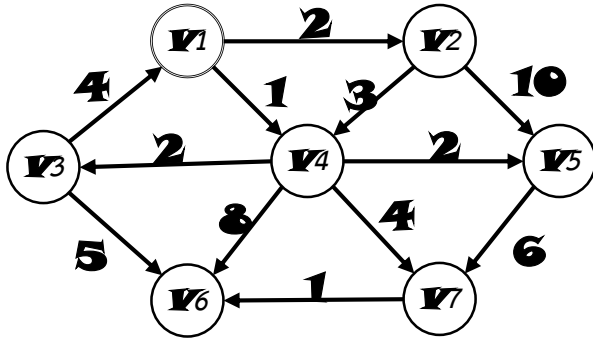
- **Dijkstra's algorithm**

- เป็น **algorithm** หนึ่งที่เป็น **greedy algorithm** ซึ่งเป็นการแก้ปัญหาเป็น **stages** โดยใช้สิ่งที่ดูเหมือนว่าดีที่สุดในแต่ละ **stage** และต้องพิสูจน์ได้ว่า ได้ผลลัพธ์ โดยรวมทั้งหมด ดีที่สุดด้วย
- **greedy algorithms** ใช้ได้ในบางกรณีเท่านั้น



- **Dijkstra's algorithm** ทำงานเป็น **stages** เช่นเดียวกับ **unweighted shortest-path algorithm**
 - ในแต่ละ **stage** จะทำการเลือก **vertex v** ตัวหนึ่งที่มีค่า d_v น้อยที่สุดในบรรดา **unknown vertices** และประกาศให้เป็นเส้นทางที่สั้นที่สุดจาก **s** ไป **v** (**known**) และ ปรับปรุงค่าของ d_w
 - ในกรณี **unweighted** เราตั้งค่าให้ $d_w = d_v + 1$ ถ้า $d_w = \infty$ นั่นคือ เราลดค่าของ d_w ลงถ้าหากว่า **vertex v** มีเส้นทางที่สั้นกว่า ถ้าเราใช้วิธีเดียวกันนี้กับกรณี **weighted** เราก็จะกำหนดให้ $d_w = d_v + c_{v,w}$ ถ้าค่าใหม่นี้ทำให้ d_w ดีขึ้น



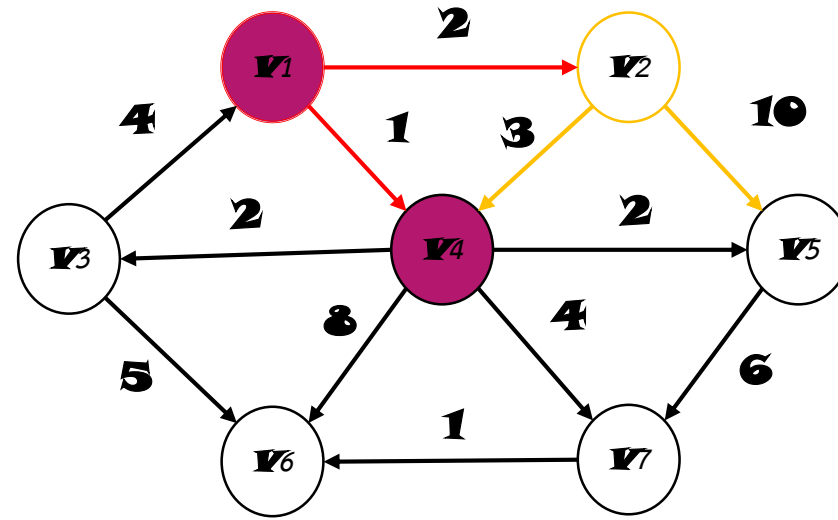
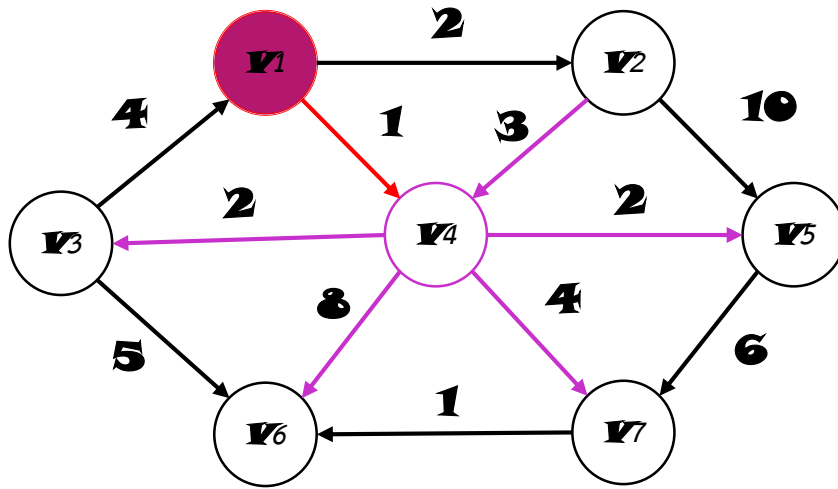


	Initial		
V	Known	d_v	p_v
V1	F	0	0
V2	F	∞	0
V3	F	∞	0
V4	F	∞	0
V5	F	∞	0
V6	F	∞	0
V7	F	∞	0

ให้ **vertex** เริ่มต้น **s**
คือ **v_1** และมีความยาวของ
เส้นทางเป็น ∞ และกำหนดให้
 v_1 เป็น **known**
ปรับ **distance v_2** และ
 v_4 ที่เป็น **adjacent**
ของ **v_1**

	After v1 known		
v	Known	d_v	p_v
v1	T	0	0
v2	F	2	v1
v3	F	∞	0
v4	F	1	v1
v5	F	∞	0
v6	F	∞	0
v7	F	∞	0





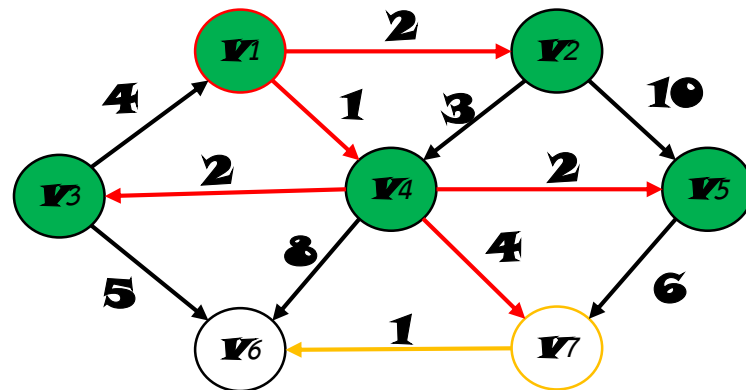
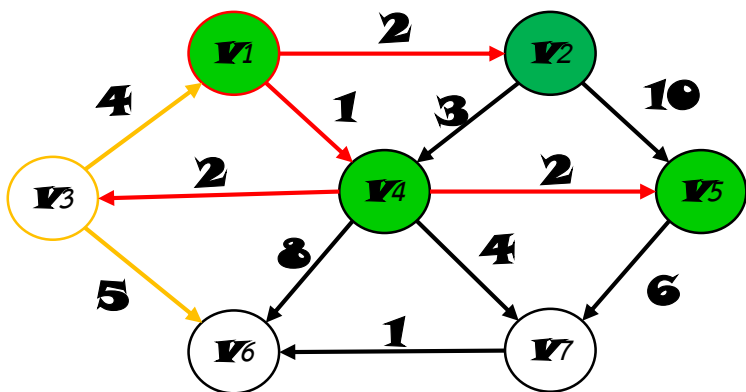
	After v4 known		
V	Known	d_v	p_v
V1	T	0	0
V2	F	2	V1
V3	F	3	V4
V4	T	1	V1
V5	F	3	V4
V6	F	9	V4
V7	F	5	V4

เลือก **v4** แล้วให้เป็น **known**

เพราะมี **distance** ต่ำสุด
ปรับปรุงค่า **V3, V5, V6** และ
V7 ที่เป็น **adjacent**

เลือก **V2** มี **V4** เป็น
adjacent แต่มันเป็น
known แล้วจึงไม่ต้องทำอะไร มี
V5 เป็น **adjacent** แต่ไม่มีการ
ปรับเปลี่ยนใด ๆ เนื่องจากระยะที่ผ่านทาง
V2 คือ **2+10 = 12** แต่ได้
รับรู้ระยะเท่ากับ **3** แล้ว

	After v2 known		
V	Known	d_v	p_v
V1	T	0	0
V2	T	2	V1
V3	F	3	V4
V4	T	1	V1
V5	F	3	V4
V6	F	9	V4
V7	F	5	V4

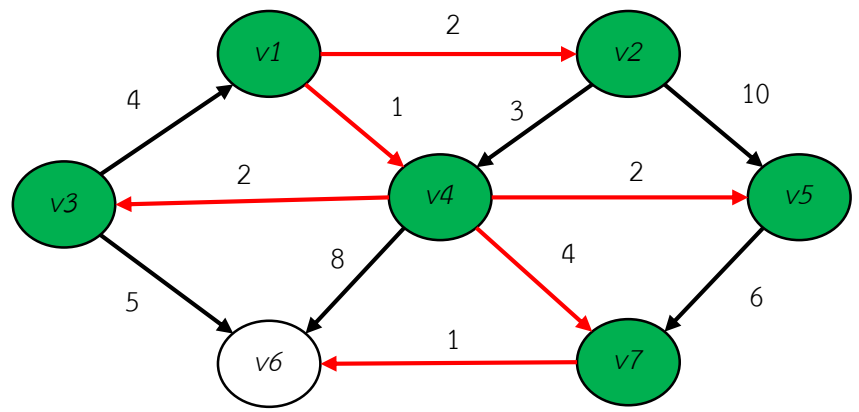


After v5 and v3 known			
V	Known	d_v	p_v
V1	T	0	0
V2	T	2	V1
V3	T	3	V4
V4	T	1	V1
V5	T	3	V4
V6	F	8	V3
V7	F	5	V4

v5 มีระยะ **3** และมี **v7** เป็น **adjacent** แต่ไม่ปรับเปลี่ยนเพราะว่า **3 + 6 > 5** จากนั้นเลือก **v3** และปรับเปลี่ยนระยะสำหรับ **v6** ลงเป็น **3 + 5 = 8**

เลือก **v7** ซึ่งต้องปรับเปลี่ยน **v6** เป็น **5+1 = 6**

After v7 known			
V	Known	d_v	p_v
V1	T	0	0
V2	T	2	V1
V3	T	3	V4
V4	T	1	V1
V5	T	3	V4
V6	F	6	V7
V7	T	5	v4



	After v6 known		
V	Known	d_v	p_v
V1	T	0	0
V2	T	2	V1
V3	T	3	V4
V4	T	1	V1
V5	T	3	V4
V6	T	6	V7
V7	T	5	v4



- พิจารณา **pseudocode** ของ **Dijkstra's algorithm**
 - แต่ละ **Vertex** เก็บ **data fields** ต่าง ๆ ที่ใช้ (**Fig. 7.29**)
 - กำหนดให้อ่านกราฟเข้าใน **array** ของ **Vertex** และสร้าง **adjacency lists** โดย **routine readGraph**
 - **Figure 7.30**, เป็น **initialization routine**
 - พิมพ์เส้นทางด้วย **recursive routine** ใน **Figure 7.31** โดยเริ่มพิมพ์ไปตามเส้นทางจนถึง **V**
 - **Figure 7.32** เป็น **main algorithm** ซึ่งเป็นเพียง **for loop** ที่ใช้ในการเพิ่มเติมข้อมูลในตาราง



```
class Vertex  
{  
    public List adj;      // Adjacent vertices  
    public boolean known;  
    public DistType dist; // DistType is probably int  
    public Vertex path; // Other fields and methods as needed  
}
```

Figure 7.29 Declarations 2023 **Dijkstra's algorithm**



```

public Vertex createTable( )
{
  /*1*/ Vertex [ ] t = readGraph( );
        // read graph somehow
  /*2*/ for( int i = 0; i < t.length; i++ )
        {
  /*3*/           t[ i ].known = false;
  /*4*/           t[ i ].dist = INFINITY;
  /*5*/           t[ i ].path = null;
        }
  /*6*/ NUM_VERTICES = t.length;
}

```

Figure 7.30 Table initialization routine



```
/* print shortest path to v after dijkstra has run */  
/* assume that the path exists */
```

```
void printPath ( Vertex v )  
{  
    if ( v.path != null )  
    {  
        printPath( v.path );  
        System.out.print ( " to " );  
    }  
    System.out.print ( v );  
}
```

Figure 7.31 Routine to print the actual shortest path



```

void dijkstra( Vertex s )
{
    vertex v, w;
    /*1*/ s.dist = 0;
    /*2*/ for( ; ; )
    {
        /*3*/ v = smallest unknown distance vertex; //minHeap
        /*4*/ if (v == null )
        /*5*/ break;
        /*6*/ v.known = true;
        /*7*/ for each w adjacent to v
        /*8*/ if ( !w.known )
        /*9*/ if ( v.dist + cvw < w.dist )
        { /* update w */
            /*10*/ decrease( w.dist to v.dist + cv,w );
            /*11*/ w.path = v;
        }
    }
}

```

Figure 7.32 Pseudocode 7.23 Dijkstra's algorithm



- **Algorithm** นี้ใช้งานได้ครบเท่าที่ไม่มี **negative cost edge** ถ้ามี **edge** ที่เป็น **negative cost** อาจให้คำตอบผิดได้

- หาก **algorithm** ที่ใช้เป็นการ **scan** ไปตามตารางเพื่อหาค่าที่น้อยที่สุด d_v เราก็จะใช้การทำงาน $O(|V|)$ หน่วยเวลาในแต่ละ **phase** การทำงาน ดังนั้นจะต้องใช้เวลาทั้งสิ้นเป็น $O(|V|^2)$ หน่วยเวลาเวลาที่ใช้ในการปรับเปลี่ยนค่า d_w เป็นเวลาคงที่ต่อการปรับเปลี่ยนแต่ละครั้ง และต้องปรับเปลี่ยนหนึ่งครั้งต่อหนึ่ง **edge** เท่านั้น ดังนั้นจำนวนการปรับเปลี่ยนค่านี้ทั้งหมดจึงเป็น $O(|E|)$ รวม **running time** ทั้งหมดของอัลกอริทึมคือ $O(|E| + |V|^2) = O(|V|^2)$



- ถ้ากราฟเป็นแบบ **dense** ที่มี $|E| = \Theta(|V|^2)$ แล้ว **algorithm** นี้ก็จะเป็น **linear** ในเทอมของจำนวน **edges**
- ถ้ากราฟเป็นแบบ **sparse** ที่มี $|E| = \Theta(|V|)$ การทำงานจะช้ามาก ในกรณีนี้จะจัดเก็บระยะทางไว้ใน **priority queue**
- ถ้าใช้โครงสร้างข้อมูลอื่น อาจจะทำให้ **time bounds** ของ **Dijkstra's algorithm** ดีขึ้นได้
 - โครงสร้างข้อมูลแบบ **Fibonacci heap** ซึ่งทำให้ **running time** เป็น $O(|E| + |V| \log |V|)$



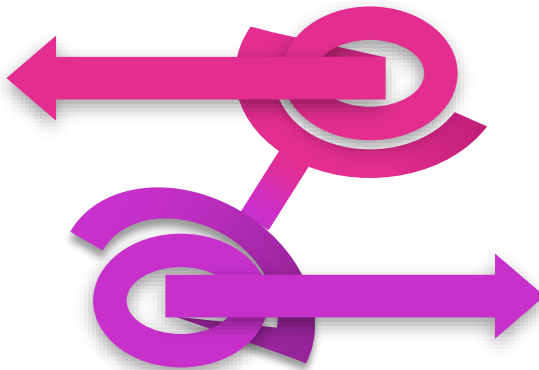
7.4. Minimum Spanning Tree

ปัญหาที่จะกล่าวถึงต่อไป คือ การหา **minimum spanning tree** ใน **undirected graph**

- **minimum spanning tree** ของ **undirected graph G** ก็คือ **tree** ที่สร้างจาก **edge** ของกราฟที่เชื่อมต่อ **vertices** ทั้งหมดของ **G** โดยที่มี **cost** รวมต่ำสุด

minimum spanning tree จะมีได้ก็ต่อเมื่อกราฟ **G** เป็นแบบ **connected** เท่านั้น

- รูปด้านล่างของ **Figure 7.48** เป็น **minimum spanning tree** ของรูปด้านบน



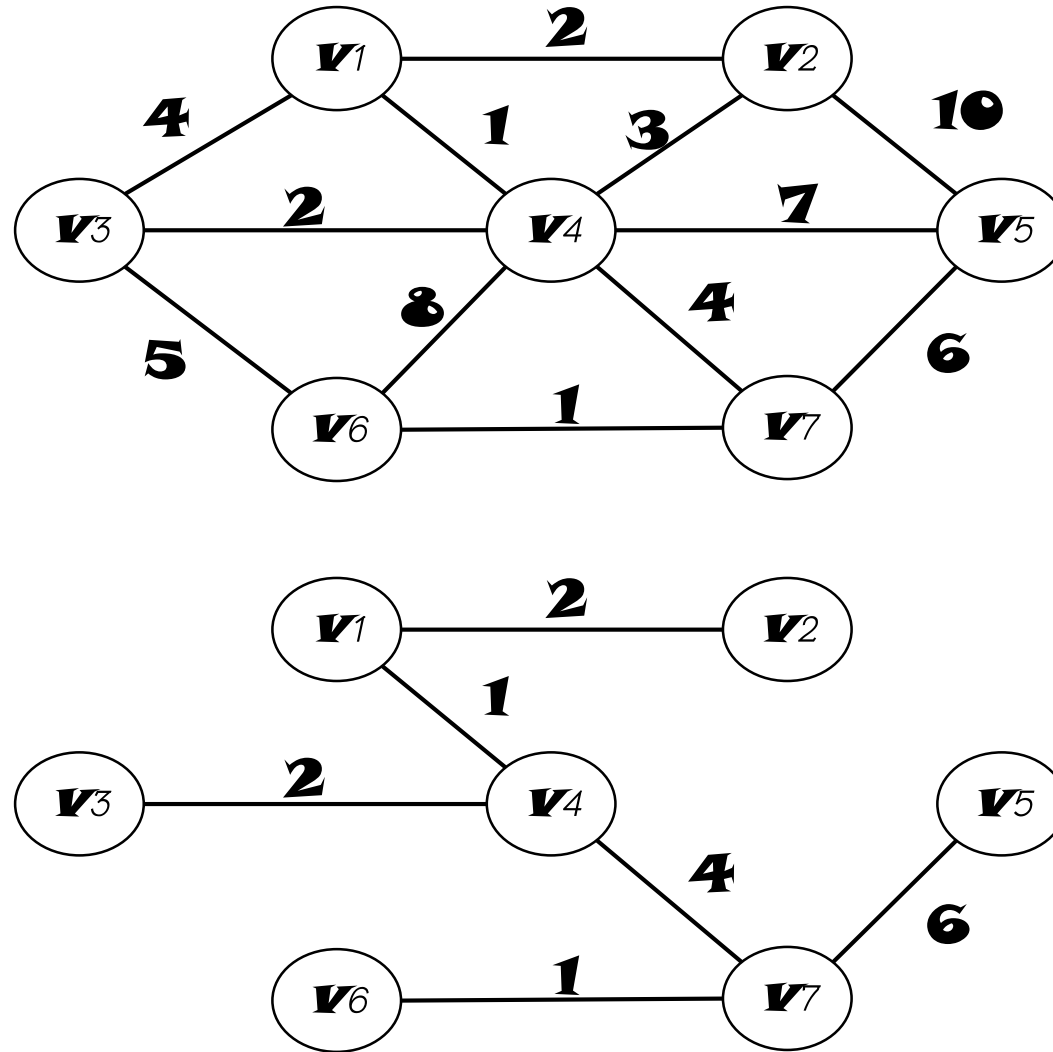


Figure 7.48 กราฟ **G** และ **minimum spanning tree** ของมัน

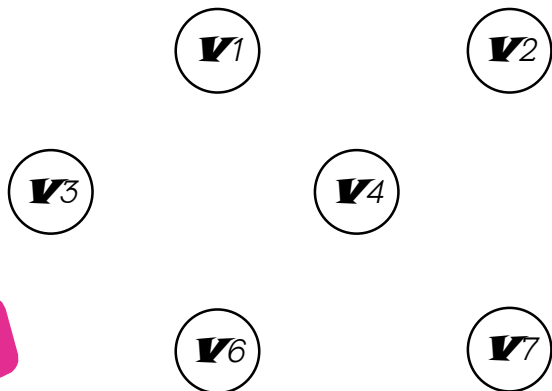
- **minimum spanning tree** เป็น
 - **tree** เนื่องจากมันเป็น **acyclic**,
 - มันเป็น **spanning** เนื่องจากมันครอบคลุมทุก **vertex**,
 - และเป็น **minimum** เนื่องจาก ผลรวมน้ำหนักของ **edges** น้อยที่สุด
- จำนวน **edges** ใน **minimum spanning tree** คือ $|V| - 1$.
- **Algorithm** พื้นฐานในการแก้ปัญหานี้มีสองแบบ และทั้งสองเป็น **greedy algorithm**



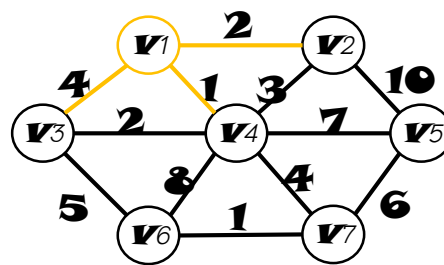
7.4.1. Prim's Algorithm

- ณ เวลาหนึ่ง เราจะมีเซตของ **vertices** ที่ได้รวมอยู่ใน **tree** แล้ว และในการทำงานแต่ละ **stage** ก็จะเป็นการค้นหา **vertex** ใหม่เพื่อที่รวมเข้ากับ **tree** ด้วยการเลือก (u, v) ที่ **cost** ของ (u, v) น้อยที่สุดในบรรดา **edges** ทั้งหมด โดยมี u อยู่ใน **tree** และมี v ไม่อยู่ใน **tree**
- **Figure 7.49** แสดงการทำงานของ **algorithm**
 - เริ่มจาก v_1 และจะมี v_1 เป็น **root** ใน **tree** โดยที่ยังไม่มี **edges**
 - แต่ละขั้นตอนการทำงานจะเป็นการเพิ่ม **edge** หนึ่งและ **vertex** หนึ่งลงใน **tree**

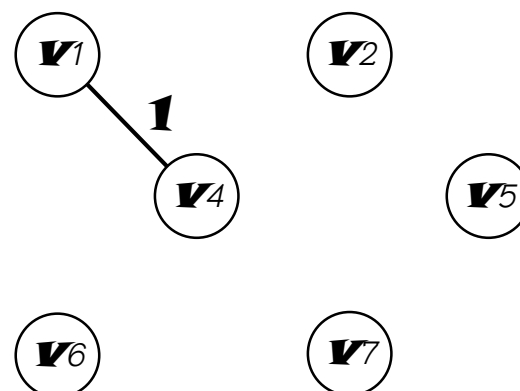




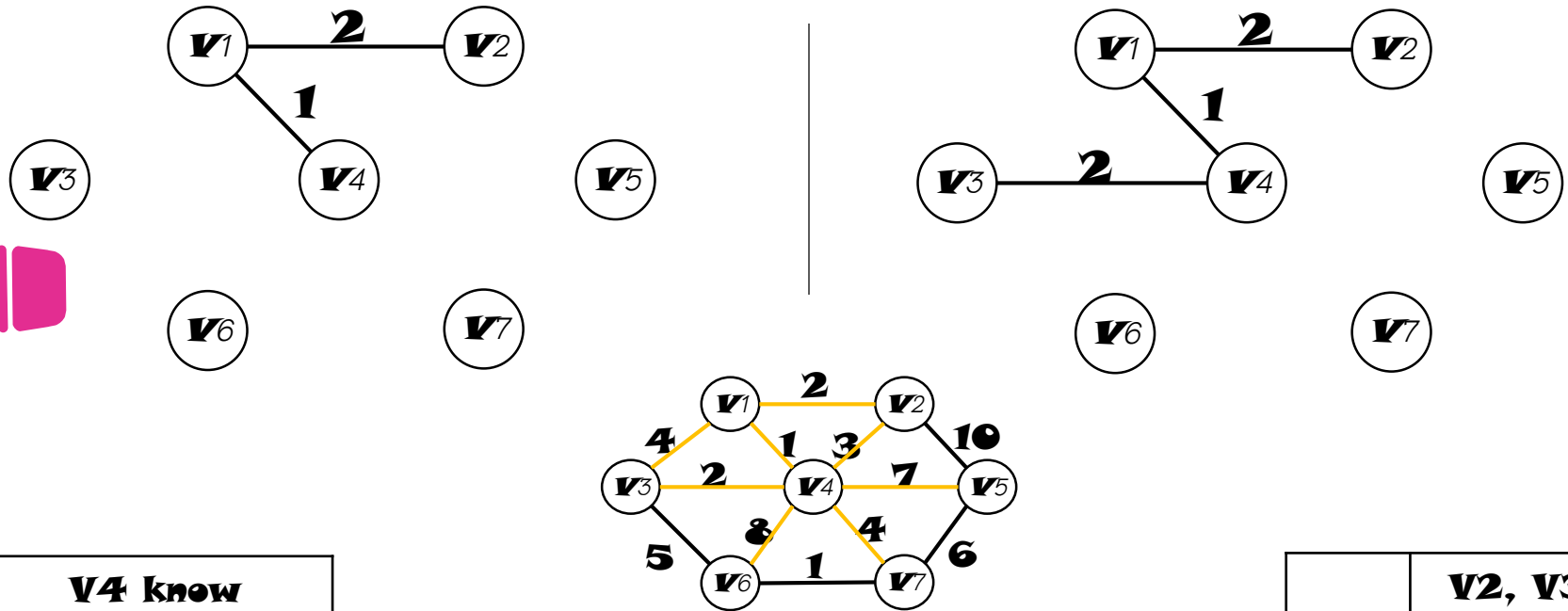
	Initial		
V	Known	d_v	p_v
V1	F	●	●
V2	F	∞	●
V3	F	∞	●
V4	F	∞	●
V5	F	∞	●
V6	F	∞	●
V7	F	∞	●



เลือก **V₁** และปรับปรุง
V₂, V₃, และ V₄



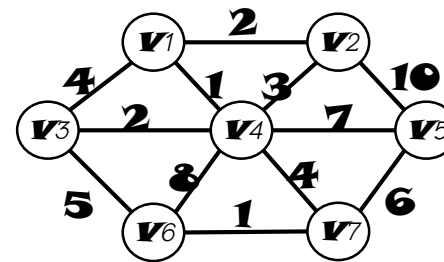
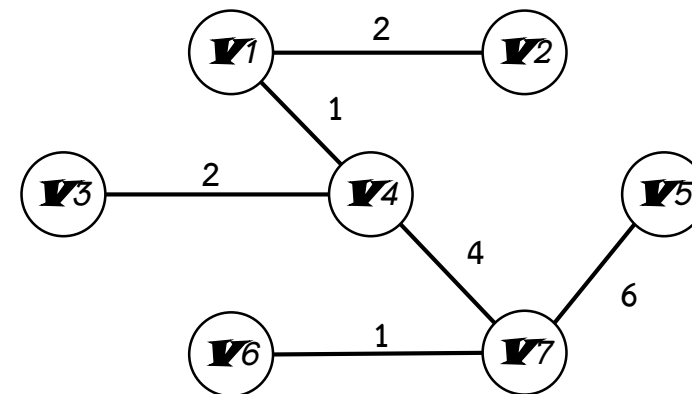
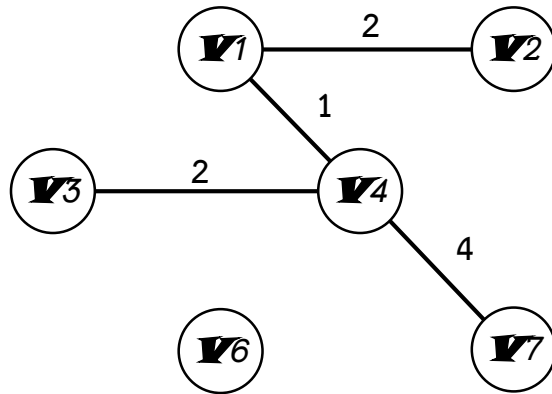
	V1 known		
V	Known	d_v	p_v
V1	T	●	●
V2	F	2	V1
V3	F	4	V1
V4	F	1	V1
V5	F	∞	●
V6	F	∞	●
V7	F	∞	●



	V4 know		
V	Known	d_v	p_v
V1	T	0	0
V2	F	2	V1
V3	F	2	V4
V4	T	1	V1
V5	F	7	V4
V6	F	8	V4
V7	F	4	V4

เลือก **v_4** เป็น **vertex** ต่อไป ทุก **vertex** เป็น **adjacent** กับ **v_4** แต่ไม่ตรวจสอบ **v_1** เนื่องจากผ่านการทำงานแล้ว (**known**) **v_2** ไม่เปลี่ยนเพราะว่ามันมี **$d_v = 2$** และ **edge cost** จาก **v_4** ไป **v_2** คือ **3** ที่เหลือนอกนั้นมี การปรับเปลี่ยน
ต่อไปเลือก **vertex v_2** ซึ่งไม่มีผลใด ๆ ต่อ ระยะทาง

	V2, V3 known		
V	Known	d_v	p_v
V1	T	0	0
V2	T	2	V1
V3	T	2	V4
V4	T	1	V1
V5	F	7	V4
V6	F	5	V3
V7	F	4	V4



	V7 known		
V	Known	d_v	p_v
V1	T	0	0
V2	T	2	V1
V3	T	2	V4
V4	T	1	V1
V5	F	6	V7
V6	F	1	V7
V7	T	4	V4

เลือก **V3** ซึ่งมีผลต่อระยะของ **V6**

เลือก **V7** ซึ่งทำให้มีการปรับเปลี่ยน **V6** และ **V5**

เลือก **V6** แล้วตามด้วย **V5**

	V6, V5 known		
V	Known	d_v	p_v
V1	T	0	0
V2	T	2	V1
V3	T	2	V4
V4	T	1	V1
V5	T	6	V7
V6	T	1	V7
V7	T	4	V4



- การทำงานของ **algorithm** นี้เหมือนกันกับ **Dijkstra's algorithm** ดังนั้นสิ่งที่กล่าวมาแล้วในการวิเคราะห์ **Dijkstra's algorithm** ก็ใช้ได้เช่นกัน
- เนื่องจาก **Prim's algorithm** ใช้สำหรับ **undirected graphs** ดังนั้นในการเขียนโปรแกรมจะต้องไม่ลืมกำหนดให้แต่ละ **edge** ปรากฏอยู่ใน **adjacency lists** สองครั้ง
- ถ้าไม่ใช้ **heap** มี **running time** เป็น $O(|V|^2)$ ซึ่งเหมาะสมกับ **dense graphs**, และเป็น $O(|E| \log |V|)$ ถ้าใช้ **binary heaps** ซึ่งเหมาะกับ **sparse graphs**



7.4.2. Kruskal's Algorithm

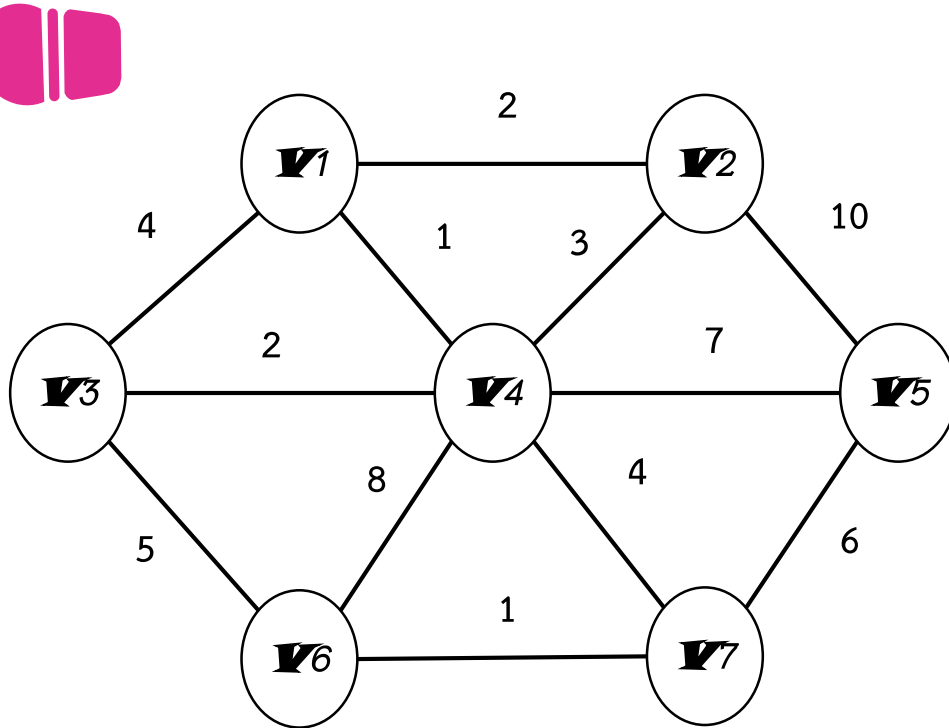
- อีกวิธีหนึ่งซึ่งเป็น **greedy algorithm** เช่นกันคือจะทำการเลือก **edges** ไปเรื่อยๆ จาก **weight** น้อยไปหามาก เพื่อให้ได้ **weight** ที่น้อยที่สุด และ ใช้ **edge** นั้นถ้ามันไม่ก่อให้เกิดการวนรอบ (**cycle**)
- กล่าวอีกนัยหนึ่ง **Kruskal's algorithm** สร้างกลุ่มของ **tree** ขึ้น เรียกว่า **forest**





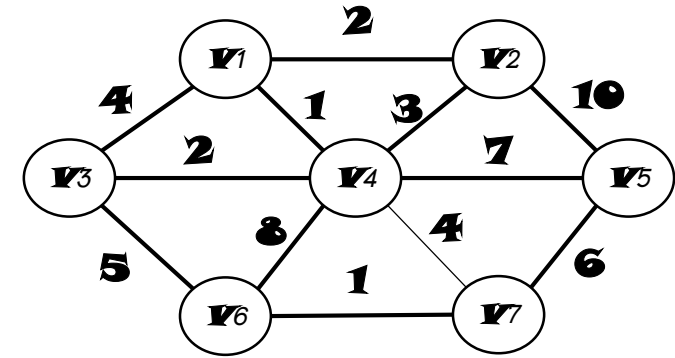
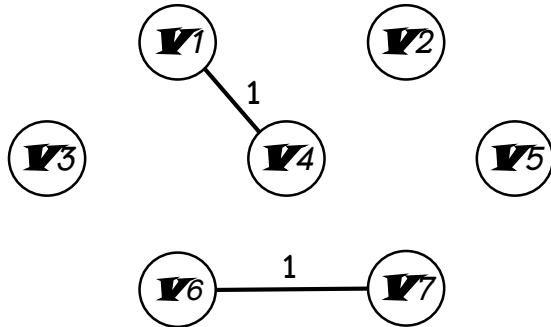
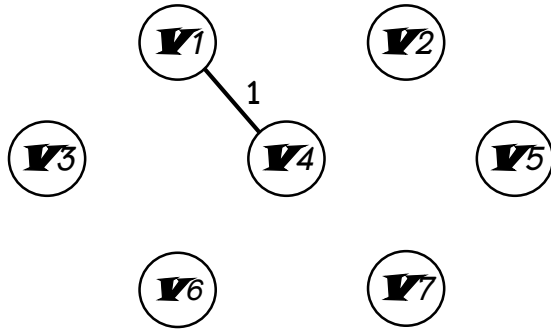
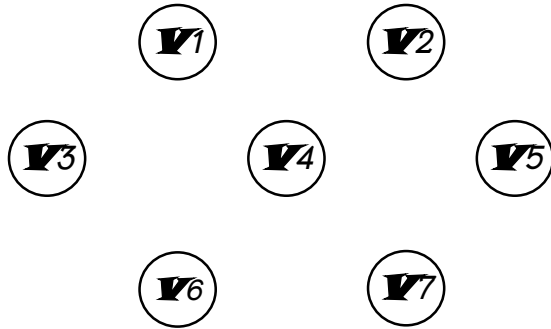
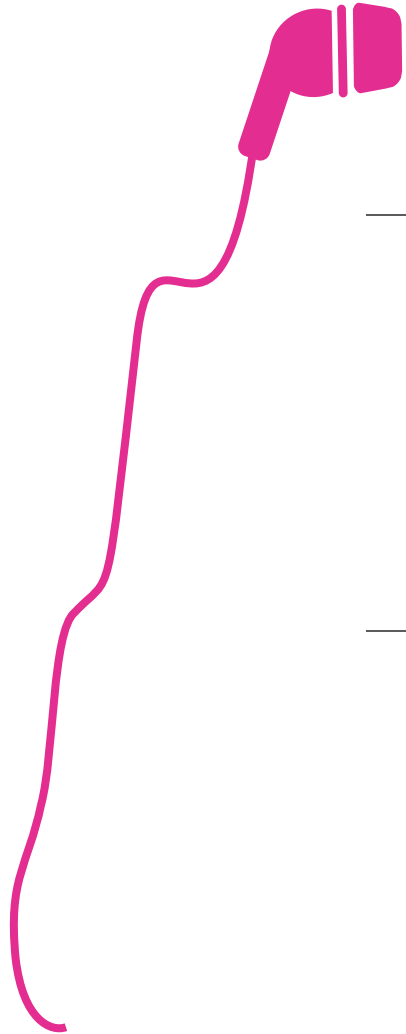
- เริ่มต้นมี **tree** ที่มี **node** เดียวจำนวน **$|V|$ trees**
- การเพิ่ม **edge** หนึ่ง เป็นการควบรวมสอง **trees** เข้าเป็น **tree** เดียว โดยเลือก **edge** ที่น้อยที่สุดในขณะนั้น
- **Tree** คือ **set** ใน **Disjoint set**
- เมื่อจบการทำงาน ก็จะมีเหลือ **tree** อยู่ **tree** เดียว และเป็น **minimum spanning tree**
- **Figure 7.57** แสดงลำดับการทำงาน



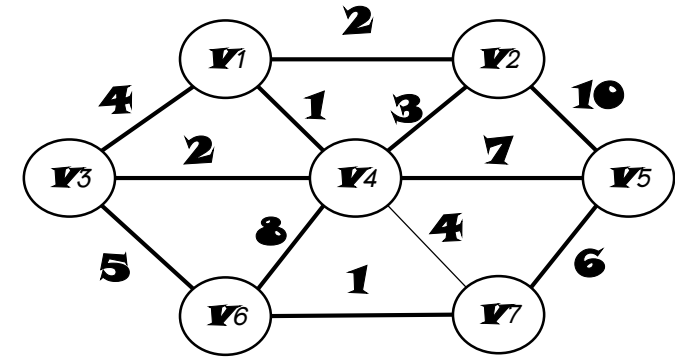
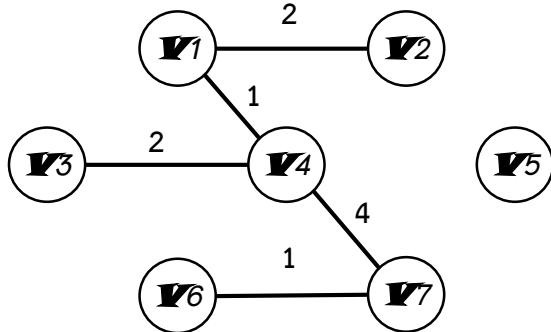
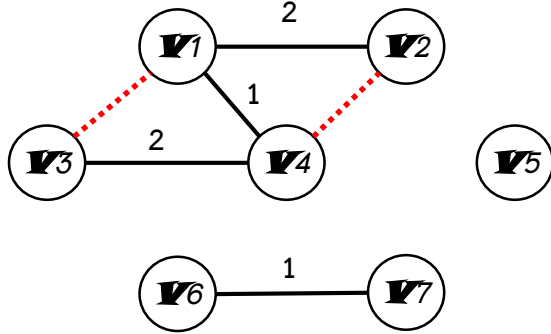
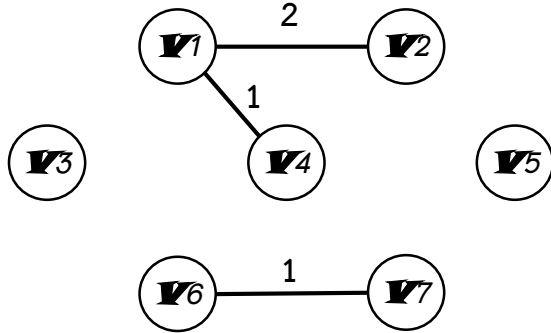
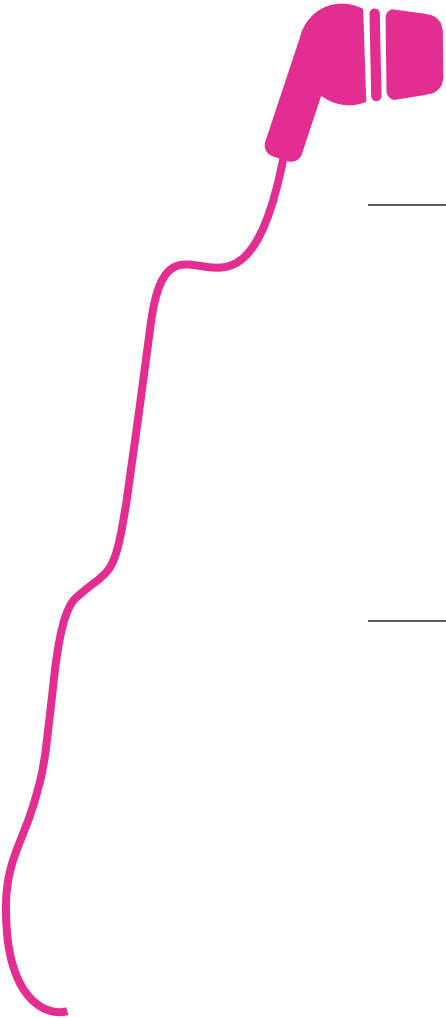


Edge	Weight	Action
V1,V4	1	
V6,V7	1	
V1,V2	2	
V3,V4	2	
V2,V4	3	
V1,V3	4	
V4,V7	4	
V3,V6	5	
V5,V7	6	

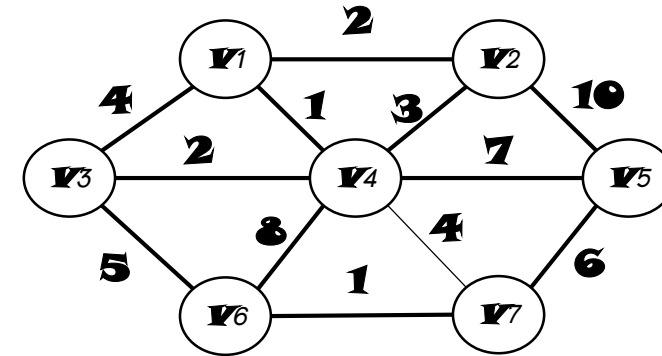
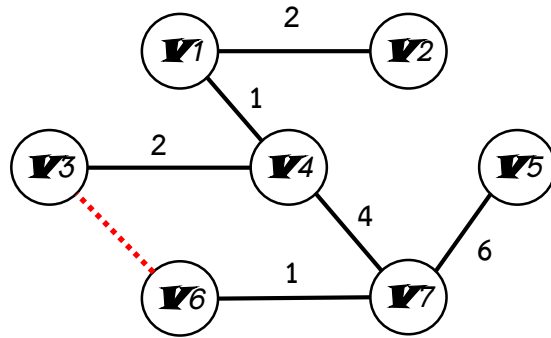
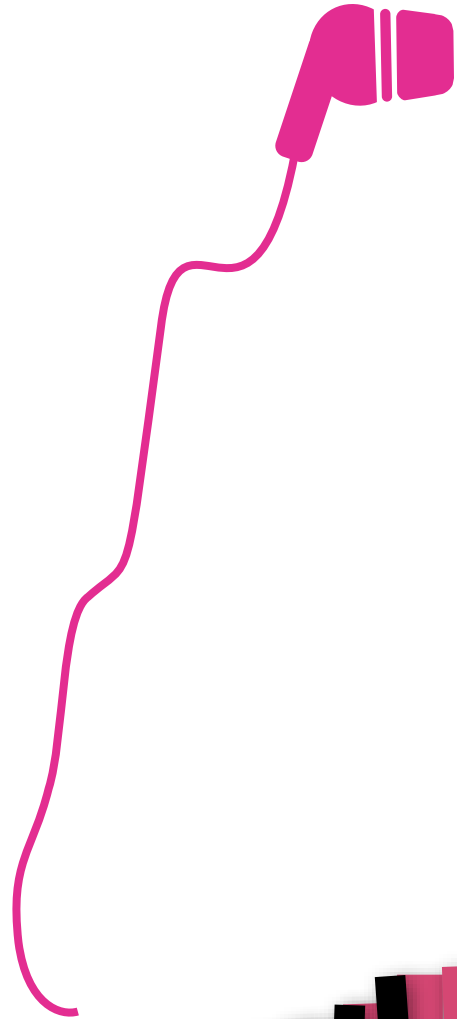
Figure 7.56 การทำงานของ **Kruskal's algorithm**



Edge	Weight	Action
V1,V4	1	Accept
V6,V7	1	Accept
V1,V2	2	
V3,V4	2	
V2,V4	3	
V1,V3	4	
V4,V7	4	
V3,V6	5	
V5,V7	6	



Edge	Weight	Action
V1,V4	1	Accept
V6,V7	1	Accept
V1,V2	2	Accept
V3,V4	2	Accept
V2,V4	3	Reject
V1,V3	4	Reject
V4,V7	4	Accept
V3,V6	5	
V5,V7	6	



Edge	Weight	Action
V1,V4	1	Accept
V6,V7	1	Accept
V1,V2	2	Accept
V3,V4	2	Accept
V2,V4	3	Reject
V1,V3	4	Reject
V4,V7	4	Accept
V3,V6	5	Reject
V5,V7	6	Accept



- **Method *kruskal*** ในรูป **Figure 7.58** ใช้ในการหา **minimum spanning tree**
- การเรียงน้ำหนัก **edge** ใช้ **heap** แล้ว **extract min**
- **worst-case running time** เป็น $O(|E| \log |E|)$ ซึ่งเป็นการทำงานของ **heap** เป็นส่วนใหญ่
- เนื่องจาก $|E| = O(|V|^2)$ ดังนั้น **running time** ก็คือ $O(|V|^2 \log |V|)$





```

public void kruskal( )
{
    int edgesAccepted;
DisjSet s;
PriorityQueue h;
Vertex u, v;
SetType uset, vset;
Edge e;
    /*1*/ h = readGraphIntoHeapArray( );
/*2*/ h.buildHeap();
/*3*/ s = new DisjSet( NUM_VERTICES );
/*4*/ edgesAccepted = 0;
/*5*/ while ( edgesAccepted < NUM_VERTICES - 1 ) {
/*6*/     e = h.deleteMin( H ); /*e = (u, v) */
/*7*/     uset = s.find( u );
/*8*/     vset = s.find( v );
/*9*/     if ( uset != vset ) {
/*10*/         /* accept the edge */
/*11*/         edgesAccepted++;
/*12*/         s.union( uset, vset );
    }
}
}

```

Figure 7.58 Pseudocode of Kruskal's algorithm



7.5 Applications of Depth-First Search

Depth-first search เป็นลักษณะทั่วไปของ **preorder traversal**

- เริ่มต้นด้วยการประมวลผลที่ **vertex v** ใด ๆ แล้วจากนั้นการจะ **traverse** ไปในทุก **vertices** แบบ **recursive** ที่เป็น **adjacent** กับ **v**
- กรณีที่มันเป็น **tree** การทำเช่นนี้ก็จะใช้เวลา $O(|E|)$ เนื่องจาก $|E| = O(|V|)$



- 
- กรณีที่มันเป็นกราฟ เราจะต้องระมัดระวังไม่ให้เกิด **cycles**
 - โดยเมื่อเราเข้าถึง **vertex v** เราจะทำเครื่องหมายไว้ แล้วเรียกใช้ **depth-first search** แบบ **recursive** กระทำต่อ **adjacent vertices** ทั้งหมดที่ยังไม่มีเครื่องหมาย
 - สำหรับ **undirected graphs** แล้วทุก ๆ **edge (v, w)** เกิดขึ้นสองครั้งใน **adjacency lists**: คือ **(v, w)** และ **(w, v)**

```
void dfs ( Vertex v )  
{  
    v.visited = true;  
    for each w adjacent to v  
        if ( !w.visited )  
            dfs( w );  
}
```



Figure 7.59 Pseudocode Depth-First Search



- ถ้ากราฟเป็นแบบ **undirected** และไม่เป็น **connected** หรือถ้าเป็นแบบ **directed** และไม่เป็นแบบ **strongly connected** วิธีการดังกล่าวข้างบนอาจล้มเหลวในการเข้าถึง **node** บาง **node** เราก็จะค้นหา **node** ที่ยังไม่มีเครื่องหมายแล้วใช้การทำงาน **depth-first traversal** กับมันต่อไป และจะทำให้เช่นนี้ไปจนกว่าจะไม่มี **node** ที่ไม่มีเครื่องหมาย
- เนื่องจากวิธีการนี้จะประกันได้ว่าจะมีการเข้าถึงแต่ละ **edge** เพียงครั้งเดียว ดังนั้นเวลาที่ใช้ในการ **traversal** คือ $O(|E| + |V|)$ เมื่อใช้ **adjacency lists**



7.5.1 Undirected Graphs



undirected graph เป็นแบบ **connected** ถ้าหากว่า
การทำ **depth-first search** โดยเริ่มจาก **node** ใด
node หนึ่งแล้วสามารถเข้าถึง **node** ได้ทุก **node**

- **Algorithm** ที่จะกล่าวถึงต่อไปจะถือว่ากราฟเป็นแบบ
connected และถ้ากราฟเป็นแบบ **unconnected** เราก็
สามารถหาส่วนที่เป็น **connected** และใช้ **algorithm**
ของเรา กับแต่ละส่วนได้





- ตัวอย่าง **depth-first search** ของกราฟรูป **Figure 8.60** โดยเริ่มที่ **vertex A** ทำเครื่องหมายที่ **A** เมื่อเข้าถึง และเรียกใช้ **dfs(B)** แบบ **recursive** ซึ่งจะทำการเครื่องหมายที่ **B** และเรียกใช้ **dfs(C)** ในแบบ **recursive** ทำเครื่องหมายที่ **C** และเรียกใช้ **dfs(D)** แบบ **recursive** ซึ่ง **dfs(D)** จะมองเห็น **A** และ **B** แต่เนื่องจากทั้งสองนี้ถูกทำเครื่องหมายแล้วจึงไม่เรียกฟังก์ชัน **recursive** อีกต่อไปและจะกลับไปยัง **dfs(C)** ซึ่ง **dfs(C)** มองเห็น **B** ก็จะถูกปล่อยไปเช่นกัน แต่จะเห็น **E** และเรียกใช้ **dfs(E)** และ.....





- เราสามารถเขียนรูป **depth-first spanning tree** เพื่อเป็นการแสดงขั้นตอนดังกล่าวข้างบนได้ดังนี้

• **Vertex** แรกที่เข้าถึง คือ **A** ให้เป็นรากของ **tree** แต่ละ **edge (v, w)** ในกราฟจะแสดงให้ปรากฏใน **tree** ถ้าขณะที่ประมวลผล **(v, w)** นั้นพบว่า **w** ยังคงเป็น **unmarked** หรือขณะที่ประมวลผล **(w, v)** พบว่า **v** ยังเป็น **unmarked** เราจะกำหนดให้ **edge** นั้นของกราฟเป็น **tree edge** ซึ่งแสดงด้วยเส้นเชื่อมต่อกติ





- ถ้าขณะประมวลผล (v, w) พบว่า w ได้รับการ **marked** แล้ว และขณะประมวลผล (w, v) พบว่า v ได้รับการ **marked** แล้ว ก็จะเขียนเป็นเส้นประ และเรียกว่า **back edge** เพื่อระบุว่า “**edge**” นี้ไม่ได้เป็นส่วนหนึ่งของ **tree**
- รูป **Figure 7.61** เป็น **depth-first search** ของกราฟในรูป **Figure 7.60**



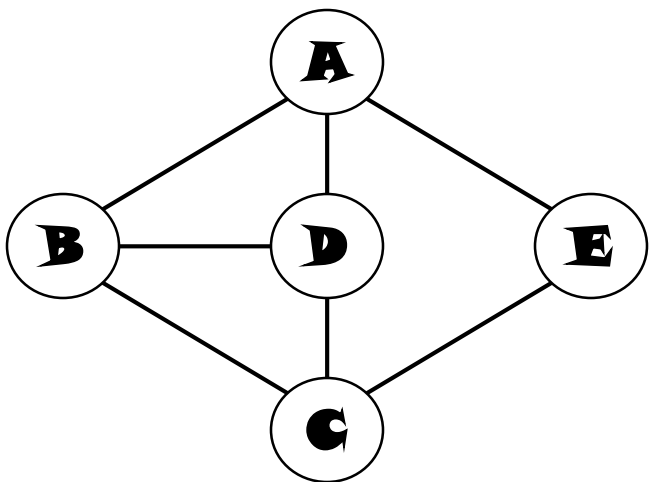
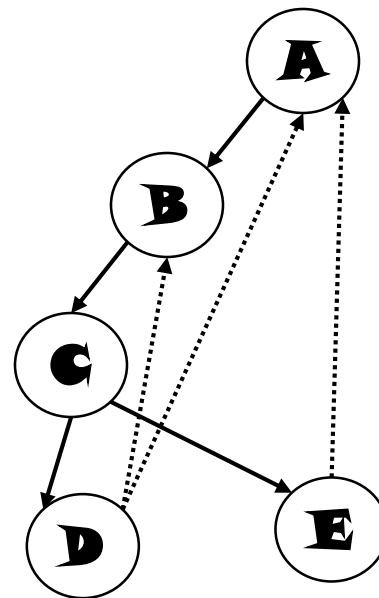


Figure 7.60 undirected graph



—— Tree edge
 Back edge

Figure 7.61 Depth-first search ของ **Figure 8.60** ตาม **A-B-C-D-E**



Thank You